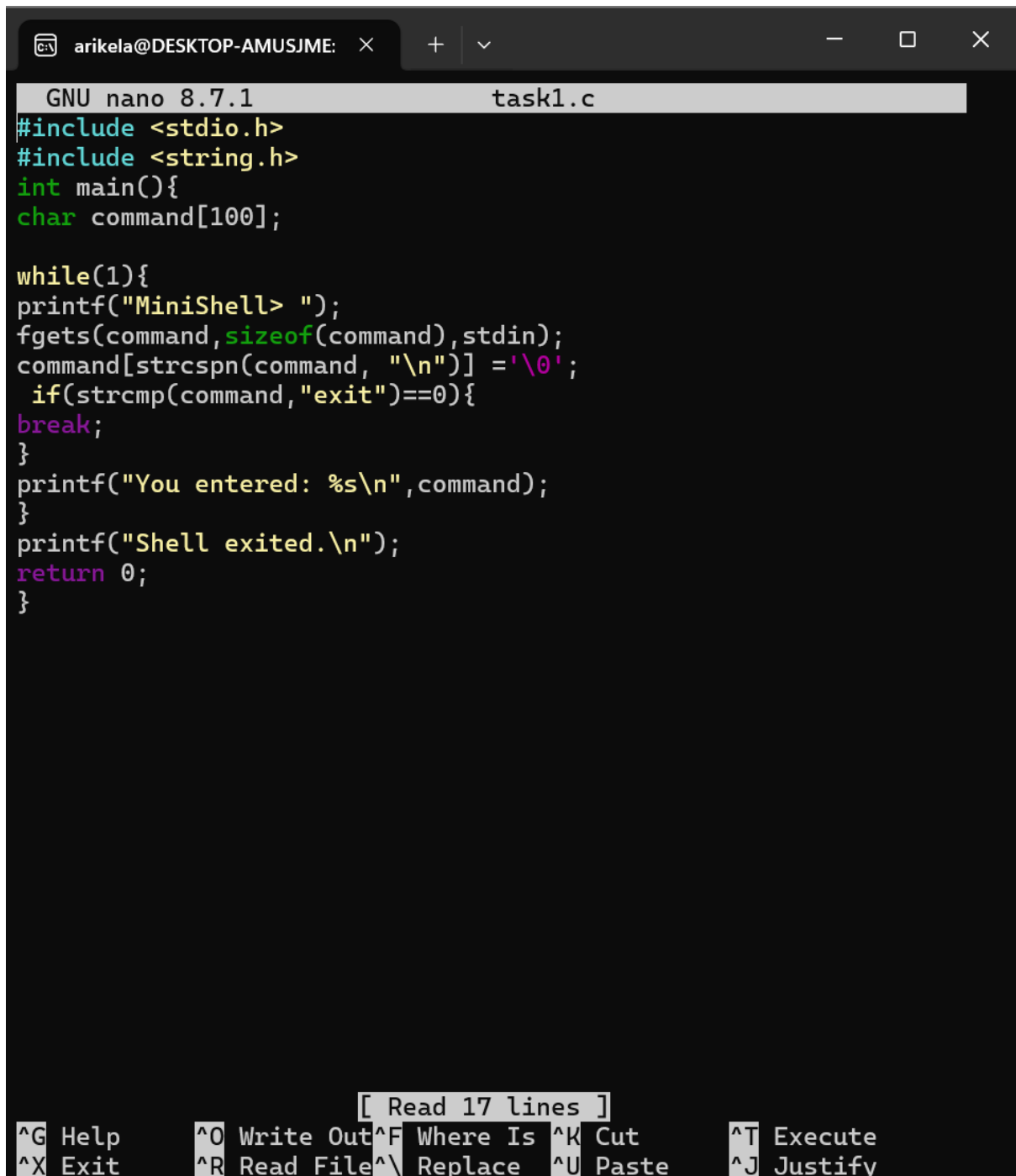


SKILL WEEK-3

TASK-1

To Create Main Loop, Display Prompt, Read User Input, Handle Exit Conditions, Design Control Flow Diagram, Test Interactive Loop



```
GNU nano 8.7.1 task1.c
#include <stdio.h>
#include <string.h>
int main(){
char command[100];

while(1){
printf("MiniShell> ");
fgets(command,sizeof(command),stdin);
command[strcspn(command, "\n")] = '\0';
if(strcmp(command,"exit")==0){
break;
}
printf("You entered: %s\n",command);
}
printf("Shell exited.\n");
return 0;
}

[ Read 17 lines ]
^G Help      ^O Write Out ^F Where Is  ^K Cut       ^T Execute
^X Exit      ^R Read File ^\ Replace   ^U Paste     ^J Justify
```

```

arikela@DESKTOP-AMUSJME:~$ nano task1.c
arikela@DESKTOP-AMUSJME:~$ gcc task1.c -o task1
arikela@DESKTOP-AMUSJME:~$ ./task1
MiniShell> hello
You entered: hello
MiniShell> ls
You entered: ls
MiniShell> pwd
You entered: pwd
MiniShell> exit
Shell exited.

```

Task-2

Using the following system calls, copy the content from File1.txt to File2.txt.

1. **Open()**
2. **Read()**
3. **Write()**
4. **Close()**

open()

syntax: open(" filename.txt", modes, permissions)

Example: open("First_Prgm.txt", O_CREAT | O_WRONLY, 0644);

Table 1: Modes of Open system call

Flag	Purpose
O_RDONLY	Open for read only
O_WRONLY	Open for write only
O_RDWR	Open for both read and write
O_CREAT	Create file if it does not exist
O_TRUNC	Truncate (empty) an existing file
O_APPEND	Append data to the end of the file
O_EXCL	Fail if file already exists (used with O_CREAT)

read()

Syntax: read(int fd, void *buf, size_t count);

Example: read(fd, buffer, 100)

```
#include <stdio.h>
```

```
#include <unistd.h>
```

```
int main()
```

```
{
```

```
    char buffer[100];
```

```
    int n;
```

```
    printf("Enter some text: ");
```

```
    n = read(0, buffer, sizeof(buffer) - 1);
```

```
    buffer[n] = '\0'; // Null-terminate the string
```

```
    printf("You entered: %s", buffer);
```

```
    return 0;
```

```
}
```

```
write()
```

Syntax: write(int fd, const void *buf, size_t count);

Example: write(fd, message, 11);

```
#include <unistd.h>
```

```
int main()
```

```
{
```

```
    write(1, "Hello World\n", 12);
```

```
    return 0;
```

```
}
```

GNU nano 8.7.1

task2.c *

```
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>

int main() {
    int fd1, fd2;
    char buffer[100];
    int n;

    fd1 = open("File1.txt", O_RDONLY);

    if (fd1 < 0) {
        printf("Error opening File1.txt\n");
        return 1;
    }

    fd2 = open("File2.txt", O_CREAT | O_WRONLY | O_TRUNC, 0644);

    if (fd2 < 0) {
        printf("Error opening File2.txt\n");
        close(fd1);
        return 1;
    }

    n = read(fd1, buffer, sizeof(buffer));

    if (n < 0) {
        printf("Error reading file\n");
        close(fd1);
        close(fd2);
        return 1;
    }
}
```

^G Help

^X Exit

^O Write Out

^R Read File

^F Where Is

^_ Replace

^K Cut

^U Paste

^T Execute

^J Justify

```

    if (n < 0) {
        printf("Error reading file\n");
        close(fd1);
        close(fd2);
        return 1;
    }

    write(fd2, buffer, n);

    close(fd1);
    close(fd2);

    printf("File copied successfully.\n");

    return 0;
}
|

```

^G Help	^O Write Out	^F Where Is	^K Cut	^T Execute
^X Exit	^R Read File	^N Replace	^U Paste	^J Justify

```

arikela@DESKTOP-AMUSJME:~$ echo "Hello, this is File1.">File1.tx
t
arikela@DESKTOP-AMUSJME:~$ cat File1.txt
Hello, this is File1.
arikela@DESKTOP-AMUSJME:~$ nano task2.c
arikela@DESKTOP-AMUSJME:~$ gcc task2.c -o task2
arikela@DESKTOP-AMUSJME:~$ ./task2
File copied successfully.
arikela@DESKTOP-AMUSJME:~$ cat File2.txt
Hello, this is File1.
arikela@DESKTOP-AMUSJME:~$ ls -l File.txt File2.txt
ls: cannot access 'File.txt': No such file or directory
File2.txt
arikela@DESKTOP-AMUSJME:~$ ls -l File1.txt File2.txt
-rw-r--r-- 1 arikela arikela 22 Aug 16 11:27 File1.txt
-rw-r--r-- 1 arikela arikela 22 Aug 16 11:33 File2.txt

```

Task-3

To Capture Keyboard Input, Handle Backspace, Process Enter Key, Manage Input Buffer, Support Multi-Character Commands, Test User Interaction.

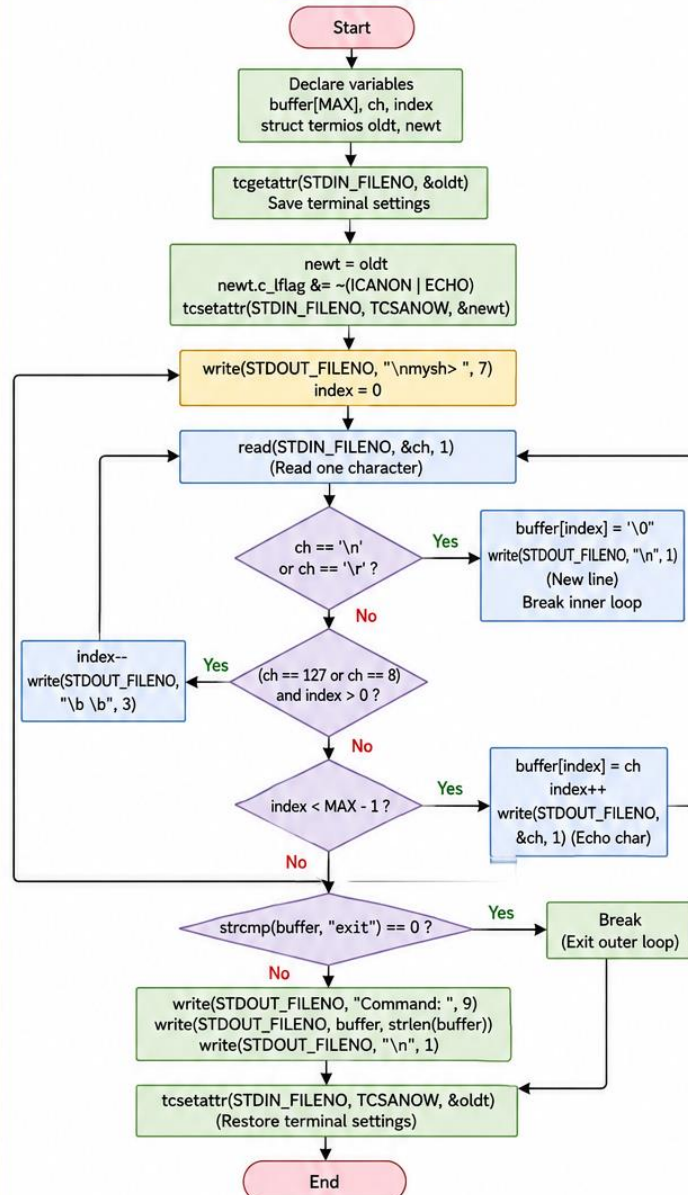
ALGORITHM & FLOWCHART

Keyboard Input Handling Using System Calls (read, write, termios)

ALGORITHM

1. Start.
2. Declare variables:
 - buffer[MAX]
 - char ch
 - int index
 - struct termios oldt, newt
3. Save the current terminal settings using `tcgetattr(STDIN_FILENO, &oldt)`.
4. Copy the terminal settings:
`newt = oldt`
5. Disable canonical mode and echo:
`newt.c_lflag &= ~(ICANON | ECHO)`
6. Apply the new terminal settings using `tcsetattr(STDIN_FILENO, TCSANOW, &newt)`.
7. Repeat forever
 - (a) Display the prompt using `write()`:
"mysh> "
 - (b) Set index = 0.
 - (c) Read characters one by one using `read()`.
 - (d) If the character is Enter ('\n' or '\r'):
 - `buffer[index] = '\0'`
 - Print new line.
 - Exit the inner loop.
 - (e) Else if the character is Backspace (ASCII 8 or 127) and index > 0:
 - `index--`
 - Print "\b \b" to remove last character.
 - (f) Else if index < MAX - 1:
 - Store the character in buffer.
 - `index++`
 - Echo the character using `write()`.
 - (g) After exiting the inner loop, compare the command in buffer.
 - If command is "exit", break the outer loop.
 - Otherwise, print "Command: " followed by the command.
8. Restore the original terminal settings using `tcsetattr(STDIN_FILENO, TCSANOW, &oldt)`.
9. End.

FLOWCHART



SYSTEM CALLS / FUNCTIONS USED

Function / System Call	Purpose
<code>read()</code>	Reads one character from keyboard (stdin).
<code>write()</code>	Writes output to terminal (stdout).
<code>tcgetattr()</code>	Gets current terminal attributes.
<code>tcsetattr()</code>	Sets terminal attributes (raw / canonical mode).
<code>strcmp()</code>	Compares strings (used to check "exit").

TERMINAL MODE

- Canonical mode (default): Input is line buffered. Backspace is handled by terminal driver.
- Non-canonical (raw) mode: Input is available character by character; program handles Backspace, Enter, etc.
- We disable ICANON (canonical) and ECHO to read characters immediately and handle them in the program.

This program uses system calls (read, write) and terminal control (termios) to capture keyboard input, handle backspace, process enter key, manage input buffer and support multi-character commands.

```
#include <stdio.h>
```

```
int main() {
    char buffer[100];
    int index = 0;
    char ch;

    printf("MiniShell> ");

    while (1) {
        ch = getchar();

        // Enter key
        if (ch == '\n') {
            buffer[index] = '\0';

            printf("\nYou entered: %s\n", buffer);

            index = 0;
            printf("MiniShell> ");
        }

        // Backspace key
        else if (ch == 127 || ch == '\b') {
            if (index > 0) {
                index--;
                printf("\b \b");
            }
        }

        // Normal character
        else {
            if (index < 99) {
```

```
^G Help
^X Exit
```

```
^O Write Out
^R Read File
```

```
^F Where Is
^_ Replace
```

```
^K Cut
^U Paste
```

```
^T Execute
^J Justify
```

```

        // Normal character
    else {
        if (index < 99) {
            buffer[index] = ch;
            index++;
            putchar(ch);
        }
    }

    return 0;
}
|

```

^G Help	^O Write Out	^F Where Is	^K Cut	^T Execute
^X Exit	^R Read File	^N Replace	^U Paste	^J Justify

```

arikel@DESKTOP-AMUSJME:~$ nano task3.c
arikel@DESKTOP-AMUSJME:~$ gcc task3.c -o task3
arikel@DESKTOP-AMUSJME:~$ ./task3
MiniShell> he llo
You entered: he llo
MiniShell>

```